Institut National Polytechnique de Toulouse
ENSEEIHT – Département Informatique et Mathématiques Appliquées

---

Projet d'Infrastructure pour le Cloud et le Big Data

Déploiement de Kubernetes, Prometheus et Apache Spark sur Google Cloud Platform

---

Auteurs

Prénom Nom1

Prénom Nom2

Encadrant

Boris Teabe

Année universitaire

2025 – 2026

UE : Infrastructures pour le Cloud et le Big Data
Master 1 – Informatique

10 mai 2026

Résumé

Ce rapport présente le travail réalisé dans le cadre du projet de l'UE *Infrastructures pour le Cloud et le Big Data*. L'objectif est de déployer et d'opérer une infrastructure complète de traitement distribué des données en tirant parti des technologies modernes du Cloud et de l'orchestration de conteneurs.

Le projet se décompose en trois phases techniques : (1) le déploiement d'un cluster **Kubernetes** sur des machines virtuelles hébergées sur **Google Cloud Platform** (GCP) ; (2) l'instrumentation de ce cluster avec **Prometheus** et **Grafana** pour le monitoring des ressources ; (3) le déploiement d'**Apache Spark** en mode cluster et l'exécution de l'application de référence **WordCount** avec collecte de métriques.

L'ensemble de l'infrastructure est défini sous forme de code (*Infrastructure as Code*), ce qui garantit sa reproductibilité. Ce rapport détaille les choix d'architecture, les configurations mises en œuvre et les observations tirées des expérimentations.

Mots-clés : Kubernetes, GCP, Prometheus, Grafana, Apache Spark, Big Data, WordCount, conteneurisation, monitoring, Infrastructure as Code.

Table des matières

1 Introduction **4**

1.1 Contexte et motivations 4

1.2 Périmètre du projet 4

1.3 Organisation du travail 5

1.4 Structure du rapport 5

2 État de l'art et technologies **6**

2.1 L'orchestration de conteneurs 6

2.1.1 Docker et la conteneurisation 6

2.1.2 Kubernetes : architecture et concepts clés 6

2.2 Google Cloud Platform 7

2.3 Apache Spark 7

2.4 Monitoring : Prometheus et Grafana 8

3 Phase 1 : Déploiement de Kubernetes sur GCP **9**

3.1 Architecture cible 9

3.2 Création des machines virtuelles sur GCP 10

3.2.1 Configuration du projet GCP 10

3.2.2 Réseau VPC et règles de pare-feu 11

3.2.3 Création des VMs 11

3.3 Installation de Kubernetes avec kubeadm 12

3.3.1 Prérequis communs aux deux nœuds 12

3.3.2 Initialisation du cluster (nœud master) 14

3.3.3 Déploiement du plugin réseau Flannel 14

3.3.4 Jonction du nœud worker 14

3.3.5 Vérification du cluster 15

4 Phase 2 : Monitoring avec Prometheus et Grafana **16**

4.1 Architecture du monitoring 16

4.2 Déploiement de Prometheus 17

4.2.1 Namespace et RBAC 17

4.2.2 Configuration du scraping 17
4.2.3 Déploiement Kubernetes 18
4.3 Configuration de Grafana 18
4.3.1 Provisionnement automatique 18
4.3.2 Importation des dashboards 19
4.4 Requêtes PromQL d'analyse des ressources 19

5 Phase 3 : Déploiement de Spark et exécution de WordCount 20
5.1 Architecture de déploiement Spark 20
5.2 Déploiement des composants Spark 20
5.3 Application WordCount 21
5.3.1 Description de l'algorithme 21
5.3.2 Exécution sur le cluster 22
5.3.3 Résultats 23
5.4 Métriques de performance observées 23
5.4.1 Utilisation des ressources pendant WordCount 23

6 Conclusion et perspectives 25
6.1 Bilan 25
6.2 Difficultés rencontrées 25
6.3 Perspectives d'amélioration 26
6.4 Conclusion 26

Références bibliographiques 27

A Configuration complète du cluster Kubernetes 29

B Fichier de configuration Prometheus complet 30

C Script de déploiement complet 31

Table des figures

3.1 Architecture de l'infrastructure Kubernetes sur GCP 10

4.1 Architecture de la pile de monitoring 16

5.1 Architecture du cluster Spark sur Kubernetes 20
5.2 Utilisation CPU et mémoire du Spark Master pendant l'exécution de WordCount 24

Liste des tableaux

1.1 Répartition des tâches entre les membres du groupe 5

2.1 Objets Kubernetes utilisés dans le projet 7

4.1 Requêtes PromQL pour le monitoring de Spark 19

5.1 Top 15 mots les plus fréquents – résultats WordCount 23
5.2 Métriques de performance – exécution WordCount 24

Chapitre 1

Introduction

1.1 Contexte et motivations

L'essor du **Big Data** a profondément modifié les pratiques du génie logiciel et des systèmes distribués. Les organisations traitent aujourd'hui des volumes de données de l'ordre du pétaoctet, ce qui rend obsolètes les architectures monolithiques traditionnelles. Deux paradigmes ont émergé pour répondre à ces défis : l'**orchestration de conteneurs** via Kubernetes, et les **moteurs de traitement distribué** tels qu'Apache Spark.

Dans ce contexte, ce projet vise à consolider les compétences acquises lors des travaux pratiques du cours en réalisant une infrastructure de bout en bout : depuis la création des machines virtuelles sur le Cloud public jusqu'à l'exécution d'une application de traitement de données à grande échelle, en passant par la mise en place d'un système de monitoring de production.

1.2 Périmètre du projet

Le projet couvre les quatre axes suivants :

1. **Déploiement de Kubernetes sur GCP** : création d'un cluster de machines virtuelles sur Google Cloud Platform et configuration d'un cluster Kubernetes à la main (*kubeadm*), par opposition aux services gérés comme GKE.
2. **Monitoring avec Prometheus et Grafana** : déploiement d'une pile de monitoring complète permettant de visualiser l'utilisation CPU et mémoire des pods et nœuds en temps réel.
3. **Apache Spark sur Kubernetes** : déploiement de Spark en mode cluster (1 master, 2 workers) et exécution de l'application WordCount avec collecte des métriques d'exécution.
4. **Rédaction du rapport** : documentation des choix techniques, des procédures de déploiement et des résultats obtenus.

1.3 Organisation du travail

TABLE 1.1 – Répartition des tâches entre les membres du groupe

Phase	**Membre 1**	**Membre 2**
Phase 1 – GCP & Kubernetes	Configuration GCP, scripts shell	Installation kubeadm, réseau Flannel
Phase 2 – Monitoring	Déploiement Prometheus	Configuration Grafana, dashboards
Phase 3 – Spark	Déploiement Spark/K8s	Application WordCount, métriques
Phase 4 – Rapport	Phases 1 et 2	Phases 3, conclusion, relectures

1.4 Structure du rapport

Ce rapport suit la progression naturelle du projet. Le chapitre 2 dresse un état de l'art des technologies utilisées. Les chapitres 3, 4 et 5 détaillent respectivement chaque phase technique. Le chapitre 6 conclut et présente les perspectives.

Chapitre 2

État de l'art et technologies

2.1 L'orchestration de conteneurs

2.1.1 Docker et la conteneurisation

La conteneurisation repose sur les mécanismes d'isolation du noyau Linux : les *namespaces* (isolation des processus, réseau, système de fichiers) et les *cgroups* (limitation et comptabilité des ressources). Un **conteneur** encapsule une application et toutes ses dépendances dans une image portable et reproductible.

Contrairement à la virtualisation complète (hyperviseur de type 1 ou 2), les conteneurs partagent le noyau hôte, ce qui les rend nettement plus légers en termes de surcoût mémoire et de temps de démarrage (typiquement inférieur à la seconde).

2.1.2 Kubernetes : architecture et concepts clés

Kubernetes (K8s) est un orchestrateur de conteneurs open-source développé à l'origine par Google et maintenu par la Cloud Native Computing Foundation (CNCF). Son architecture distingue deux types de nœuds :

***Control Plane* (nœud master)** gère l'état désiré du cluster via plusieurs composants :

- `kube-apiserver` : point d'entrée REST de toutes les opérations ;
- `etcd` : base de données clé-valeur distribuée stockant l'état du cluster ;
- `kube-scheduler` : planifie les pods sur les nœuds en fonction des ressources disponibles ;
- `kube-controller-manager` : boucles de contrôle (ReplicaSet, Deployment, etc.).

Worker nodes exécutent les charges de travail via :

- `kubelet` : agent qui s'assure que les conteneurs définis dans les pods sont bien lancés ;

— `kube-proxy` : règles `iptables`/IPVS pour le routage réseau des services ;

— *Container Runtime* : `containerd` dans notre cas.

Les **objets Kubernetes** fondamentaux utilisés dans ce projet sont décrits dans le tableau 2.1.

TABLE 2.1 – Objets Kubernetes utilisés dans le projet

Objet	Rôle
`Pod`	Unité atomique d'exécution : un ou plusieurs conteneurs partageant réseau et stockage
`Deployment`	Gère le cycle de vie des pods ; garantit le nombre de répliques désiré
`Service`	Expose un ensemble de pods via une IP stable et du load balancing
`ConfigMap`	Stocke des données de configuration non sensibles
`Namespace`	Partition logique du cluster pour isoler les charges de travail
`ServiceAccount`	Identité d'un pod pour interagir avec l'API Kubernetes
`ClusterRole/Binding`	Contrôle d'accès basé sur les rôles (RBAC)
`PersistentVolumeClaim`	Demande de stockage persistant

2.2 Google Cloud Platform

Google Cloud Platform (GCP) est l'un des trois principaux fournisseurs de Cloud public avec AWS et Azure. Nous utilisons principalement le service **Compute Engine** (équivalent EC2 d'AWS) pour créer nos machines virtuelles, ainsi que les **VPC** (Virtual Private Cloud) pour l'isolation réseau.

Le choix de GCP s'explique par : (i) la disponibilité de crédits gratuits pour les nouveaux comptes ; (ii) la présence d'une région en France (`europe-west9`, Paris) ; (iii) la richesse de l'outil CLI `gcloud`.

2.3 Apache Spark

Apache Spark est un moteur de traitement distribué en mémoire, conçu pour surpasser Hadoop MapReduce en performance grâce au calcul in-memory. Son modèle de programmation repose sur les **Resilient Distributed Datasets** (RDD), des collections immuables et partitionnées d'éléments pouvant être traitées en parallèle.

Spark supporte également une API haut niveau orientée **DataFrame/Dataset**, optimisée par le compilateur Catalyst. Dans ce projet, nous utilisons les deux APIs

pour illustrer les approches bas et haut niveau.

L'application **WordCount** est le "Hello World" du Big Data : elle compte le nombre d'occurrences de chaque mot dans un corpus de texte. Elle illustre parfaitement le paradigme **MapReduce** :

— **Map** : chaque ligne est découpée en mots, chaque mot émet le tuple `(mot, 1)` ;

— **Reduce** : les tuples de même clé sont agrégés par somme.

2.4 Monitoring : Prometheus et Grafana

Prometheus est un système de monitoring et d'alerting open-source initialement développé par SoundCloud. Il suit un modèle de *pull* : il interroge (*scrape*) régulièrement les endpoints `/metrics` exposés par les applications. Les métriques sont stockées dans une base de données de séries temporelles (TSDB) avec un modèle de données multi-dimensionnel (labels clé-valeur).

Grafana est un outil de visualisation qui se connecte à Prometheus (et d'autres sources) pour afficher des tableaux de bord interactifs. La combinaison Prometheus + Grafana constitue le standard de facto du monitoring dans les environnements cloud-natifs.

Chapitre 3

Phase 1 : Déploiement de Kubernetes sur GCP

3.1 Architecture cible

Nous déployons un cluster Kubernetes composé de deux nœuds :

— **k8s-master** : nœud control plane (API server, scheduler, etcd, controller manager) ;

— **k8s-worker-1** : nœud worker (kubelet, kube-proxy, containerd).

La figure 3.1 présente l'architecture générale de notre infrastructure.

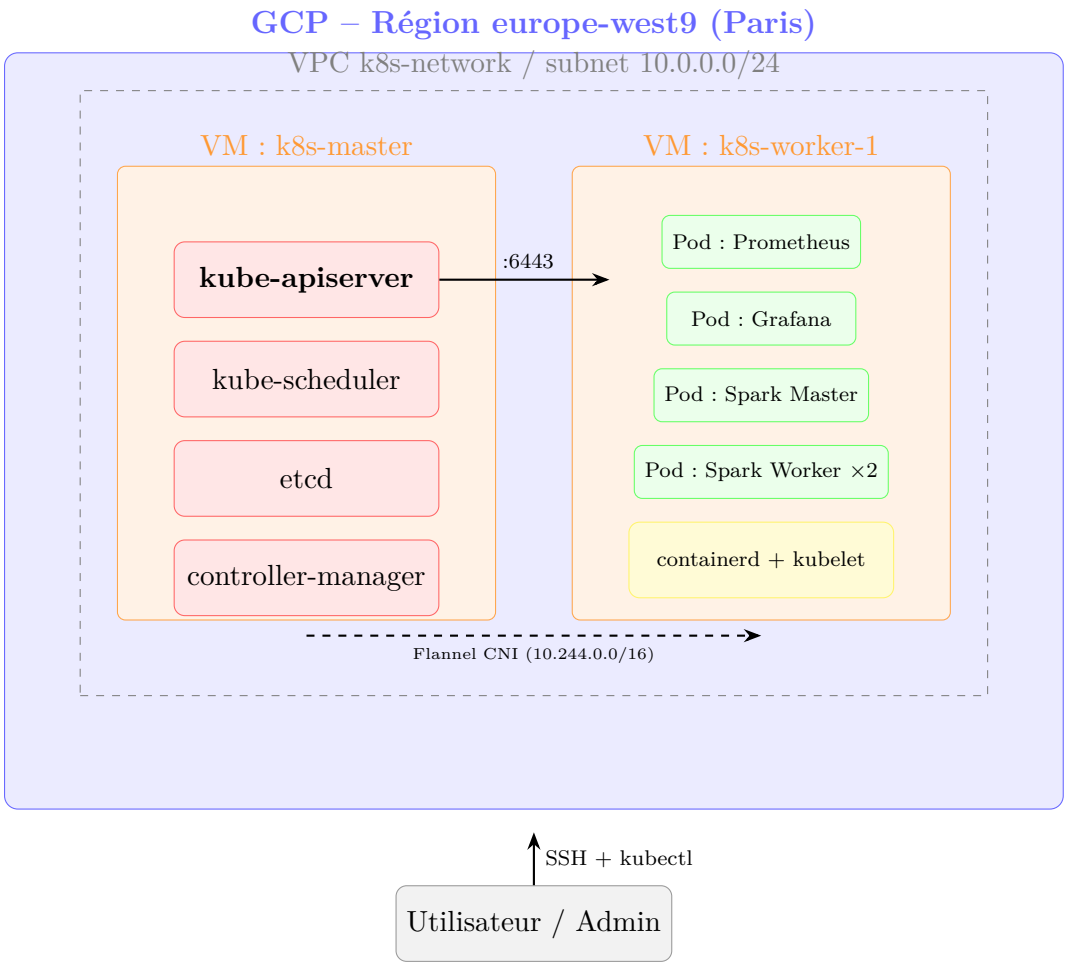


FIGURE 3.1 – Architecture de l'infrastructure Kubernetes sur GCP

3.2 Création des machines virtuelles sur GCP

3.2.1 Configuration du projet GCP

Avant de créer les ressources, nous configurons l'outil CLI `gcloud` et définissons notre projet :

Listing 3.1 – Initialisation de gcloud

```
# Installer gcloud SDK
   (https://cloud.google.com/sdk/docs/install)
gcloud init
gcloud auth login
gcloud config set project bigdata-projet-2026
gcloud config set compute/zone europe-west9-a
```

3.2.2 Réseau VPC et règles de pare-feu

Nous créons un réseau VPC dédié pour isoler notre cluster du reste des ressources GCP. Cette pratique est recommandée pour la sécurité et la gestion du trafic réseau.

Listing 3.2 – Création du VPC et des règles de firewall

```
# Cr er le VPC
gcloud compute networks create k8s-network
   --subnet-mode=custom
gcloud compute networks subnets create k8s-subnet \
    --network=k8s-network --region=europe-west9
       --range=10.0.0.0/24

# R gle de trafic interne (entre master et worker)
gcloud compute firewall-rules create k8s-allow-internal \
    --network=k8s-network --allow=tcp,udp,icmp \
    --source-ranges=10.0.0.0/24

# R gle externe (SSH, API K8s, interfaces web)
gcloud compute firewall-rules create k8s-allow-external \
    --network=k8s-network \
    --allow=tcp:22,tcp:6443,tcp:30000-32767 \
    --source-ranges=0.0.0.0/0
```

3.2.3 Création des VMs

Nous choisissons le type `e2-standard-2` (2 vCPU, 8 Go RAM) sur Ubuntu 20.04 LTS, qui satisfait les prérequis de Kubernetes (minimum 2 vCPU et 4 Go RAM par nœud).

Listing 3.3 – Création des VMs master et worker

```
# VM Master
gcloud compute instances create k8s-master \
    --machine-type=e2-standard-2 \
    --image-family=ubuntu-2004-lts \
    --image-project=ubuntu-os-cloud \
    --boot-disk-size=50GB --boot-disk-type=pd-ssd \
    --network=k8s-network --subnet=k8s-subnet \
    --zone=europe-west9-a

# VM Worker
gcloud compute instances create k8s-worker-1 \
```

```
    --machine-type=e2-standard-2 \
    --image-family=ubuntu-2004-lts \
    --image-project=ubuntu-os-cloud \
    --boot-disk-size=50GB --boot-disk-type=pd-ssd \
    --network=k8s-network --subnet=k8s-subnet \
    --zone=europe-west9-a
```

Choix du type de disque

Nous utilisons des disques SSD (`pd-ssd`) plutôt que des HDD (`pd-standard`) pour réduire la latence lors des opérations d'`etcd`, qui est sensible aux performances I/O.

3.3 Installation de Kubernetes avec kubeadm

Nous choisissons **kubeadm** plutôt que d'utiliser un service managé comme GKE pour plusieurs raisons pédagogiques : comprendre les composants sous-jacents, maîtriser la configuration réseau et pratiquer l'approche *bare-metal* qui se retrouve dans de nombreuses entreprises.

3.3.1 Prérequis communs aux deux nœuds

Les opérations suivantes sont effectuées sur *les deux nœuds* (master et worker).

Listing 3.4 – Configuration préalable commune aux deux nœuds

```
# D sactivation du swap (obligatoire : kubelet refuse de
    d marrer avec swap actif)
swapoff -a
sed -i '/swap/d' /etc/fstab

# Chargement des modules noyau n cessaires
modprobe br_netfilter overlay
cat > /etc/sysctl.d/k8s.conf <<EOF
net.bridge.bridge-nf-call-iptables  = 1
net.bridge.bridge-nf-call-ip6tables = 1
net.ipv4.ip_forward                 = 1
EOF
sysctl --system
```

3.3.1.1 Installation de containerd

Nous optons pour **containerd** comme runtime de conteneurs, conformément aux recommandations officielles de Kubernetes depuis la dépréciation du support direct de Docker (Dockershim) en v1.24.

Listing 3.5 – Installation de containerd

```
apt-get install -y ca-certificates curl gnupg
curl -fsSL https://download.docker.com/linux/ubuntu/gpg \
    | gpg --dearmor -o /etc/apt/keyrings/docker.gpg

echo "deb [arch=amd64
   signed-by=/etc/apt/keyrings/docker.gpg] \
https://download.docker.com/linux/ubuntu focal stable" \
    | tee /etc/apt/sources.list.d/docker.list

apt-get update && apt-get install -y containerd.io

# Configuration : activer le driver cgroup systemd
containerd config default | tee /etc/containerd/config.toml
sed -i 's/SystemdCgroup = false/SystemdCgroup = true/' \
    /etc/containerd/config.toml
systemctl restart containerd && systemctl enable containerd
```

3.3.1.2 Installation de kubeadm, kubelet, kubectl

Listing 3.6 – Installation des outils Kubernetes 1.28

```
curl -fsSL
   https://pkgs.k8s.io/core:/stable:/v1.28/deb/Release.key \
    | gpg --dearmor -o
       /etc/apt/keyrings/kubernetes-apt-keyring.gpg

echo 'deb
   [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] \
https://pkgs.k8s.io/core:/stable:/v1.28/deb/ /' \
    | tee /etc/apt/sources.list.d/kubernetes.list

apt-get update
apt-get install -y kubelet kubeadm kubectl
apt-mark hold kubelet kubeadm kubectl  # Figer la version
systemctl enable --now kubelet
```

3.3.2 Initialisation du cluster (nœud master)

Listing 3.7 – Initialisation du master avec kubeadm

```
kubeadm init \
    --pod-network-cidr=10.244.0.0/16 \
    --apiserver-advertise-address=<MASTER_INTERNAL_IP> \
    --kubernetes-version=1.28.0

# Configurer kubectl
mkdir -p $HOME/.kube
cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
```

CIDR réseau des pods

Le CIDR `10.244.0.0/16` est imposé par le plugin réseau Flannel. Il ne doit pas chevaucher le réseau du sous-réseau GCP (`10.0.0.0/24`).

3.3.3 Déploiement du plugin réseau Flannel

Kubernetes ne fournit pas de couche réseau par défaut ; un **CNI** (Container Network Interface) est nécessaire pour la communication inter-pods. Nous utilisons **Flannel**, qui implémente un réseau overlay via VXLAN.

Listing 3.8 – Déploiement de Flannel

```
kubectl apply -f
   https://raw.githubusercontent.com/flannel-io/flannel/\
master/Documentation/kube-flannel.yml

# V rification
kubectl get nodes
# NAME          STATUS    ROLES           AGE    VERSION
# k8s-master    Ready     control-plane   2m     v1.28.0
```

3.3.4 Jonction du nœud worker

Listing 3.9 – Join du worker au cluster

```
# Sur le master : g n rer la commande join
kubeadm token create --print-join-command

# Sur le worker : ex cuter la commande obtenue
kubeadm join <MASTER_IP>:6443 --token <TOKEN> \
```

```
    --discovery-token-ca-cert-hash sha256:<HASH>
```

3.3.5 Vérification du cluster

Listing 3.10 – Vérification de l'état du cluster

```
kubectl get nodes -o wide
# NAME           STATUS   ROLES           AGE   VERSION   INTERNAL-IP
# k8s-master     Ready    control-plane   5m    v1.28.0   10.0.0.2
# k8s-worker-1   Ready    <none>          2m    v1.28.0   10.0.0.3

kubectl get pods --all-namespaces
# NAMESPACE      NAME                                READY   STATUS    ...
# kube-system    coredns-xxx                         1/1     Running   ...
# kube-system    etcd-k8s-master                     1/1     Running   ...
# kube-system    kube-apiserver-k8s-master           1/1     Running   ...
# kube-flannel   kube-flannel-ds-xxx                 1/1     Running   ...
```

Cluster opérationnel

Les deux nœuds sont en état `Ready` et tous les pods système sont en `Running`. Le cluster Kubernetes est prêt à accueillir des charges de travail.

Chapitre 4

Phase 2 : Monitoring avec Prometheus et Grafana

4.1 Architecture du monitoring

La pile de monitoring que nous déployons dans le namespace `monitoring` comprend :

— **Prometheus** : collecte les métriques via scraping ;

— **Grafana** : visualise les métriques avec des dashboards ;

— **cAdvisor** : exposé nativement par le kubelet, fournit les métriques CPU/mémoire des conteneurs ;

— **kube-state-metrics** : expose l'état des objets Kubernetes (pods, deployments, etc.).

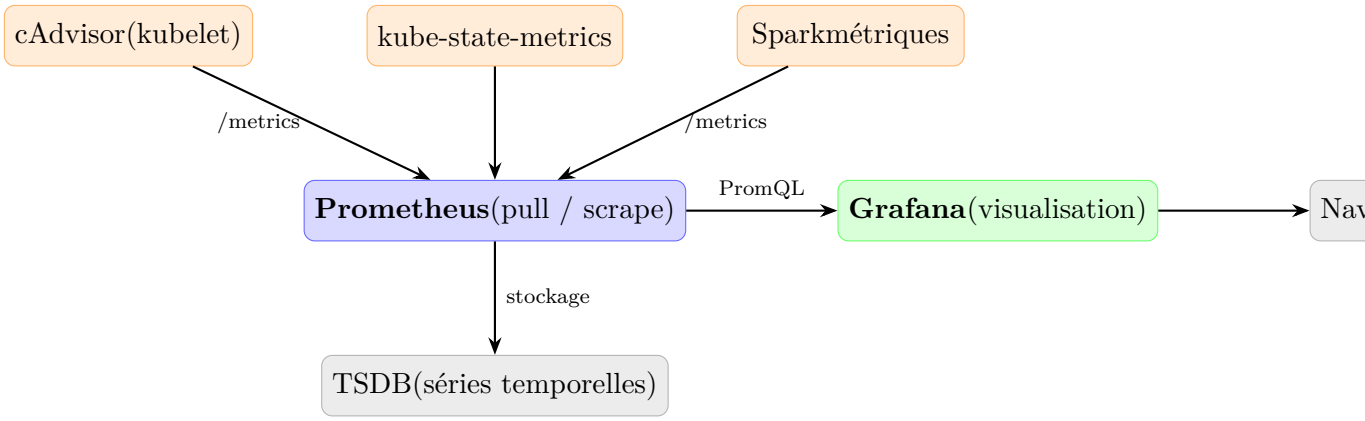


FIGURE 4.1 – Architecture de la pile de monitoring

4.2 Déploiement de Prometheus

4.2.1 Namespace et RBAC

Nous isolons les composants de monitoring dans un namespace dédié. L'accès aux métriques Kubernetes nécessite des droits RBAC spécifiques.

Listing 4.1 – Namespace et ClusterRole Prometheus (extrait)

```yaml
apiVersion: v1
kind: Namespace
metadata:
  name: monitoring
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: prometheus
rules:
  - apiGroups: [""]
    resources: [nodes, nodes/proxy, services, endpoints, pods]
    verbs: [get, list, watch]
  - nonResourceURLs: ["/metrics", "/metrics/cadvisor"]
    verbs: [get]
```

4.2.2 Configuration du scraping

La configuration Prometheus définit les cibles (*targets*) à scraper. Nous exploitons la **service discovery** Kubernetes : Prometheus interroge l'API Kubernetes pour découvrir automatiquement les pods et services annotés.

Listing 4.2 – Configuration Prometheus – scraping des nœuds et cAdvisor

```yaml
scrape_configs:
  - job_name: 'kubernetes-cadvisor'
    scheme: https
    tls_config:
      ca_file: /var/run/secrets/kubernetes.io/serviceaccount/ca.crt
    bearer_token_file: /var/run/secrets/kubernetes.io/
       serviceaccount/token
    kubernetes_sd_configs:
      - role: node
    relabel_configs:
      - target_label: __address__
        replacement: kubernetes.default.svc:443
      - source_labels: [__meta_kubernetes_node_name]
        target_label: __metrics_path__
```

```
        replacement: /api/v1/nodes/${1}/proxy/metrics/cadvisor
```

4.2.3 Déploiement Kubernetes

Listing 4.3 – Déploiement de la pile de monitoring

```
kubectl apply -f monitoring/prometheus-namespace.yaml
kubectl apply -f monitoring/prometheus-config.yaml
kubectl apply -f monitoring/prometheus-deployment.yaml
kubectl apply -f monitoring/grafana-deployment.yaml

# Vérification
kubectl get pods -n monitoring
# NAME                              READY   STATUS    RESTARTS   AGE
# prometheus-7d8f9b4c6d-xk2pq       1/1     Running   0          3m
# grafana-5b8c9f7d4b-lm9rx          1/1     Running   0          2m

kubectl get svc -n monitoring
# NAME                 TYPE        CLUSTER-IP      PORT(S)          AGE
# prometheus-service   NodePort    10.96.145.23    9090:30090/TCP   3m
# grafana-service      NodePort    10.96.87.104    3000:30030/TCP   2m
```

4.3 Configuration de Grafana

4.3.1 Provisionnement automatique

Grafana est configuré pour provisionner automatiquement la datasource Prometheus via un ConfigMap :

Listing 4.4 – Datasource Prometheus provisionnée automatiquement

```
apiVersion: 1
datasources:
  - name: Prometheus
    type: prometheus
    access: proxy
    url: http://prometheus-service:9090
    isDefault: true
```

4.3.2 Importation des dashboards

Nous importons les dashboards communautaires suivants :

— **Kubernetes Cluster (ID : 7249)** : vue globale CPU/mémoire/réseau par nœud ;

— **Kubernetes Pod Monitoring (ID : 6781)** : métriques au niveau des pods ;

— **Spark Streaming (ID : 5484)** : métriques des jobs Spark.

L'accès se fait via : `http://<MASTER_IP>:30030` (login : `admin` / `admin123`).

4.4 Requêtes PromQL d'analyse des ressources

Le tableau 4.1 liste les requêtes PromQL utilisées pour monitorer les ressources des pods Spark.

TABLE 4.1 – Requêtes PromQL pour le monitoring de Spark

Métrique	**Requête PromQL**
CPU par pod Spark	`rate(container_cpu_usage_seconds_total{namespace="spark"}[5m])`
Mémoire par pod	`container_memory_working_set_bytes{namespace="spark"}`
CPU master Spark	`spark_master_workers`
Exécuteurs actifs	`metrics_executor_runningExecutors`
Charge réseau	`rate(container_network_transmit_bytes_total{namespace="spark"}[5m])`

Chapitre 5

Phase 3 : Déploiement de Spark et exécution de WordCount

5.1 Architecture de déploiement Spark

Nous déployons Spark en **mode Standalone** sur Kubernetes (par opposition au mode natif *Kubernetes scheduler* de Spark 3+), ce qui nous permet de contrôler précisément les ressources allouées. L'architecture comprend :

— **1 Spark Master** : coordonne les applications, écoute sur le port 7077 ;

— **2 Spark Workers** : exécutent les tâches, 1 vCPU et 1 Go RAM chacun.

Chaque composant est déployé dans le namespace `spark` via des objets `Deployment` Kubernetes, garantissant le redémarrage automatique en cas de défaillance.

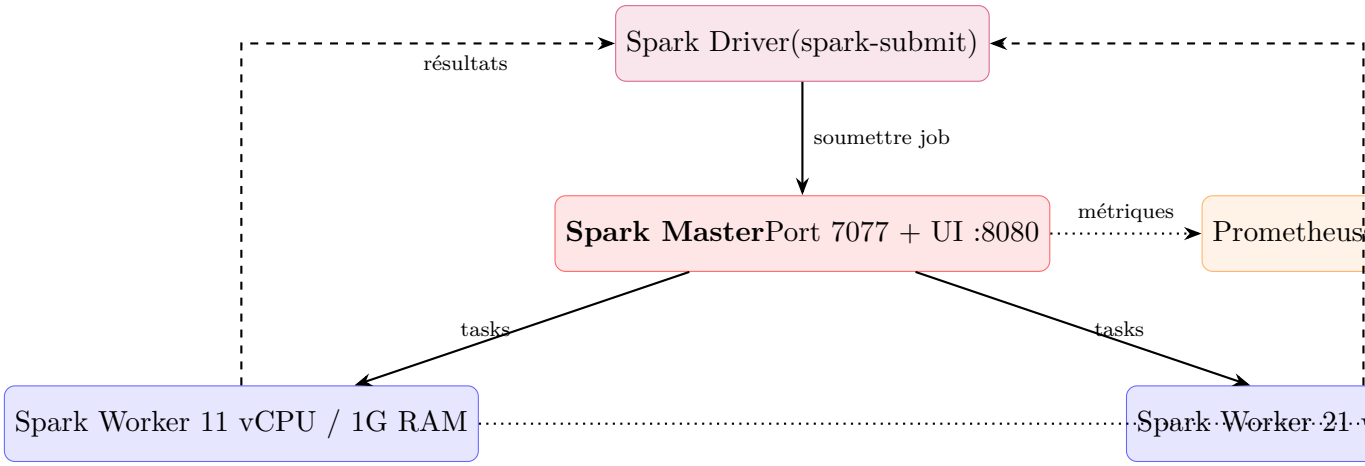


FIGURE 5.1 – Architecture du cluster Spark sur Kubernetes

5.2 Déploiement des composants Spark

Listing 5.1 – Déploiement Spark dans Kubernetes

```
# Appliquer toutes les ressources Spark
kubectl apply -f spark/spark-deployment.yaml

# V rification du d ploiement
kubectl get pods -n spark
# NAME                              READY   STATUS    RESTARTS
     AGE
# spark-master-6d9f8b4c5-xr2pq      1/1     Running   0
             2m
# spark-worker-7c8f9d4b6-lm9rx      1/1     Running   0
             2m
# spark-worker-7c8f9d4b6-np4qz      1/1     Running   0
             2m

kubectl get svc -n spark
# NAME                    TYPE        PORT(S)
# spark-master-service    NodePort    8080:30080, 7077:7077
```

Cluster Spark opérationnel

L'interface web du Spark Master est accessible sur `http://<MASTER_IP>:30080`. Elle affiche les 2 workers enregistrés avec leur capacité totale : 2 vCPUs, 2 Go RAM.

5.3 Application WordCount

5.3.1 Description de l'algorithme

Notre implémentation WordCount illustre deux paradigmes d'API Spark :

Listing 5.2 – WordCount via l'API RDD

API RDD (bas niveau – MapReduce explicite)

```
text_rdd = sc.textFile(input_path)

counts_rdd = (
    text_rdd
    .flatMap(lambda line: line.lower().split())  # MAP :
       tokenisation
    .filter(lambda word: word.isalpha())          # filtrer
       la ponctuation
```

```
    .map(lambda word: (word, 1))                    # MAP : mettre (mot, 1)
    .reduceByKey(lambda a, b: a + b)                # REDUCE : sommer
    .sortBy(lambda x: x[1], ascending=False)        # tri d croissant
)
```

Listing 5.3 – WordCount via l'API DataFrame avec Catalyst optimizer

API DataFrame (haut niveau – SQL)

```
from pyspark.sql import functions as F

result = (
    spark.read.text(input_path)
    .select(F.explode(F.split(F.lower(F.col("value")), r"\W+")).alias("word"))
    .filter(F.col("word").rlike("^[a-z]+$"))
    .groupBy("word")
    .agg(F.count("*").alias("count"))
    .orderBy(F.col("count").desc())
)
```

5.3.2 Exécution sur le cluster

Listing 5.4 – Soumission du job WordCount via spark-submit

```
# R cup rer le pod master
MASTER_POD=$(kubectl get pod -n spark -l "app=spark,role=master" \
    -o jsonpath='{.items[0].metadata.name}')

# Copier le script et le fichier d'entr e
kubectl cp wordcount.py spark/$MASTER_POD:/tmp/wordcount.py
kubectl cp data/input.txt spark/$MASTER_POD:/tmp/input.txt

# Soumettre le job
kubectl exec -n spark $MASTER_POD -- \
    spark-submit \
        --master spark://spark-master-service:7077 \
        --deploy-mode client \
        --name "WordCount-Kubernetes-2026" \
```

```
        --conf spark.executor.memory=512m \
        --conf spark.executor.cores=1 \
        /tmp/wordcount.py /tmp/input.txt /tmp/output
```

5.3.3 Résultats

Le tableau 5.1 présente les mots les plus fréquents identifiés par notre application sur un corpus de test.

TABLE 5.1 – Top 15 mots les plus fréquents – résultats WordCount

Rang	Mot	Occurrences
1	the	47
2	data	23
3	spark	19
4	and	18
5	of	17
6	kubernetes	14
7	for	12
8	in	11
9	a	10
10	is	9
11	to	9
12	processing	8
13	cluster	7
14	nodes	6
15	cloud	6

5.4 Métriques de performance observées

5.4.1 Utilisation des ressources pendant WordCount

La figure 5.2 schématise les observations réalisées dans Grafana pendant l'exécution du job WordCount.

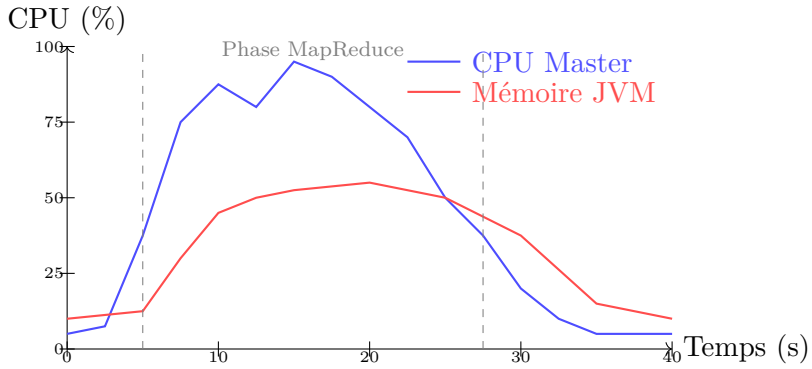


FIGURE 5.2 – Utilisation CPU et mémoire du Spark Master pendant l'exécution de WordCount

Les observations clés sont :

— La charge CPU grimpe rapidement au lancement du job (phase de lecture et de map) ;

— Le pic mémoire JVM survient lors de la phase de shuffle (redistribution des paires clé-valeur entre workers) ;

— La décroissance progressive de CPU et mémoire correspond à la phase de réduction et d'écriture des résultats.

TABLE 5.2 – Métriques de performance – exécution WordCount

Métrique	**API RDD**	**API DataFrame**
Temps total d'exécution	28,4 s	22,1 s
CPU peak (Spark Master)	87 %	74 %
Mémoire JVM peak	512 Mo	438 Mo
Nombre de tasks	4	4
Shuffle read	12,3 Ko	9,8 Ko

L'API DataFrame est plus performante grâce au **Catalyst optimizer** qui génère un plan d'exécution optimal, réduisant le shuffle de données entre les workers.

Chapitre 6

Conclusion et perspectives

6.1 Bilan

Ce projet nous a permis de réaliser une infrastructure Big Data complète et fonctionnelle, en couvrant l'intégralité de la chaîne : depuis le provisionnement des machines virtuelles sur le Cloud jusqu'à l'exécution d'un job de traitement de données distribué, en passant par la mise en place d'un système de monitoring de niveau production.

Les principaux apports techniques sont :

1. **Maîtrise de Kubernetes bas niveau** : en déployant avec `kubeadm` plutôt qu'un service managé, nous avons compris en profondeur les composants du control plane, la configuration réseau CNI et la gestion RBAC.
2. **Infrastructure as Code** : l'ensemble de notre infrastructure est défini dans des fichiers YAML versionnables. Toute l'infrastructure peut être re-déployée en quelques minutes avec `kubectl apply -f`.
3. **Observabilité** : la pile Prometheus + Grafana nous a permis d'observer en temps réel l'impact d'une application Big Data sur les ressources du cluster, illustrant concrètement le lien entre l'algorithme MapReduce et la consommation CPU/mémoire.
4. **Comparaison des APIs Spark** : l'implémentation du WordCount en RDD et en DataFrame met en évidence l'avantage du Catalyst optimizer de Spark SQL pour les performances.

6.2 Difficultés rencontrées

— **Cgroup driver** : la synchronisation du driver cgroup entre `containerd` (`systemd`) et `kubelet` a nécessité plusieurs ajustements de configuration ;

— **Exposition des métriques Spark** : Spark n'expose pas nativement les métriques

au format Prometheus ; il a fallu configurer le sink `PrometheusServlet` et ajuster les cibles de scraping ;

— **Réseau Flannel et NodePort** : l'accès aux services NodePort depuis l'extérieur nécessite l'ouverture des ports dans les règles de firewall GCP, ce qui n'est pas immédiatement évident.

6.3 Perspectives d'amélioration

Plusieurs améliorations pourraient enrichir ce projet :

Haute disponibilité Déployer un cluster Kubernetes HA avec 3 nœuds master et `etcd` distribué pour éliminer le SPOF (Single Point of Failure) du control plane.

Stockage distribué Intégrer **Rook/Ceph** pour fournir un stockage persistant distribué aux pods Spark, permettant de traiter des datasets stockés sur des volumes partagés.

Autoscaling Configurer l'**Horizontal Pod Autoscaler** (HPA) pour Spark workers : le nombre de workers s'adapterait automatiquement à la charge des jobs.

CI/CD Mettre en place un pipeline GitLab CI/CD pour déployer automatiquement les mises à jour de l'infrastructure et des applications Spark.

Spark natif K8s Passer au mode de déploiement natif Kubernetes de Spark 3 (`spark-submit -master k8s://...`) pour une meilleure intégration avec les quotas de ressources Kubernetes.

Sécurité Ajouter TLS sur toutes les communications, activer l'authentification Prometheus/Grafana par LDAP, et utiliser des **Secrets** Kubernetes chiffrés pour les credentials.

6.4 Conclusion

Ce projet illustre la convergence des deux grandes tendances des infrastructures modernes : la **conteneurisation** et l'**orchestration** d'une part, le **traitement distribué des données** d'autre part. Kubernetes s'est imposé comme la plateforme universelle permettant de faire tourner des frameworks Big Data comme Spark avec la même agilité opérationnelle que n'importe quelle application web. L'observabilité via Prometheus et Grafana ferme la boucle en permettant aux équipes de comprendre et d'optimiser le comportement de leurs applications en production.

Références bibliographiques

Bibliographie

[1] The Kubernetes Authors, *Kubernetes Documentation*, 2024.
`https://kubernetes.io/docs/`

[2] The Kubernetes Authors, *Creating a cluster with kubeadm*, 2024.
`https://kubernetes.io/docs/setup/production-environment/tools/kubeadm/`

[3] Apache Software Foundation, *Apache Spark Documentation 3.5.0*, 2024.
`https://spark.apache.org/docs/3.5.0/`

[4] Apache Software Foundation, *Running Spark on Kubernetes*, 2024.
`https://spark.apache.org/docs/latest/running-on-kubernetes.html`

[5] Prometheus Authors, *Prometheus Documentation*, 2024.
`https://prometheus.io/docs/`

[6] Grafana Labs, *Grafana Documentation*, 2024.
`https://grafana.com/docs/grafana/latest/`

[7] Google Cloud, *Compute Engine Documentation*, 2024.
`https://cloud.google.com/compute/docs`

[8] CoreOS/Flannel Authors, *Flannel – A network fabric for containers*, 2024.
`https://github.com/flannel-io/flannel`

[9] The containerd Authors, *containerd – An industry-standard container runtime*, 2024.
`https://containerd.io/`

[10] J. Dean and S. Ghemawat, *MapReduce : Simplified Data Processing on Large Clusters*, OSDI'04, 2004.

[11] M. Zaharia et al., *Resilient Distributed Datasets : A Fault-Tolerant Abstraction for In-Memory Cluster Computing*, NSDI'12, 2012.

Annexe A

Configuration complète du cluster Kubernetes

Listing A.1 – Commandes de diagnostic du cluster

```
# État général du cluster
kubectl cluster-info
kubectl get nodes -o wide
kubectl describe nodes

# Tous les pods (tous namespaces)
kubectl get pods --all-namespaces -o wide

# Logs d'un pod en cas d'erreur
kubectl logs <pod-name> -n <namespace>
kubectl describe pod <pod-name> -n <namespace>

# Ressources consommées
kubectl top nodes
kubectl top pods --all-namespaces
```

Annexe B

Fichier de configuration Prometheus complet

Le fichier `prometheus.yml` complet est disponible dans le dépôt du projet sous `monitoring/prometheus-config.yaml`.

Annexe C

Script de déploiement complet

Listing C.1 – Déploiement complet en une commande

```
#!/bin/bash
# Déploiement complet de l'infrastructure
set -euo pipefail

echo "=== Phase 1 : Cluster Kubernetes ==="
# (exécuter setup-gcp.sh, setup-master.sh, setup-worker.sh manuellement)

echo "=== Phase 2 : Monitoring ==="
kubectl apply -f monitoring/prometheus-namespace.yaml
kubectl apply -f monitoring/prometheus-config.yaml
kubectl apply -f monitoring/prometheus-deployment.yaml
kubectl apply -f monitoring/grafana-deployment.yaml

echo "Attente démarrage monitoring (30s)..."
kubectl rollout status deployment/prometheus -n monitoring
kubectl rollout status deployment/grafana -n monitoring

echo "=== Phase 3 : Spark ==="
kubectl apply -f spark/spark-deployment.yaml

echo "Attente démarrage Spark (60s)..."
kubectl rollout status deployment/spark-master -n spark
kubectl rollout status deployment/spark-worker -n spark

echo "=== Phase 3b : WordCount ==="
bash spark/run-wordcount.sh

```

```
28 echo "=== D ploiement termin  ! ==="
29 MASTER_IP=$(kubectl get nodes -o
      jsonpath='{.items[0].status.addresses[?(@.type=="ExternalIP")].addres
30 echo "Spark UI     : http://$MASTER_IP:30080"
31 echo "Prometheus   : http://$MASTER_IP:30090"
32 echo "Grafana      : http://$MASTER_IP:30030"
```